CSCI 430 Project 2 Implementation Stage 1

Person 2 Class Designs

Group 9

Abel Asfaw, Connor Doolittle, Muhammad Irfan Younas, Ethan Manninen

Person 2: Abel Asfaw Due October 25, 2026

In this part I design ClientMenuState, the menu used while a single client is active. I built it on Person 1's WarehouseContext and WarehouseState, and I modeled it on Userstate from the LibraryFSM sample (samplecode/Userstate.java). Just like Userstate works on the member ID stored in LibContext, my client menu works only on the client ID stored in WarehouseContext. Warehouse and the other Project 1 backend classes remain unchanged.

ClientMenuState extends WarehouseState

ClientMenuState corresponds to Userstate in the LibraryFSM. It is reached either by a direct client login from OpeningState or by Become a client from ClerkMenuState. Every operation uses context.getClientID(), so the menu never asks the user for a client ID. Just like in the library FSM, the state does not know which menu comes next; it only asks Context to log out.

Fields

private static ClientMenuState state; Singleton client menu, the same pattern as the static userstate field in the LibraryFSM's Userstate.

private Warehouse warehouse; Facade obtained through context.getWarehouse() when the menu runs. Userstate calls Library.instance() in its constructor; I fetch it in run() instead so the singleton does not touch Context during construction (Person 1's rule).

Command constants (private static final int): LOGOUT = 0, SHOW_DETAILS = 1, SHOW_PRODUCTS = 2, SHOW_TRANSACTIONS = 3, ADD_TO_WISHLIST = 4, DISPLAY_WISHLIST = 5, PLACE_ORDER = 6, HELP = 7. As in the library FSM, 0 leaves the menu and HELP is the largest command, so getCommand can check the range LOGOUT through HELP. These are local menu numbers, not Context events.

I do not keep a BufferedReader or a copy of the client ID in this class. Userstate has its own reader and getToken; I use the one shared context.getToken from Person 1 so all menus read System.in the same way.

Construction and input helpers

private ClientMenuState(); Calls super() only. It does not call WarehouseContext.instance().

public static ClientMenuState instance(); Returns the shared client menu, creating it on first use.

private int getCommand(); Reads a command with context.getToken("Enter command: " + HELP + " for help"). Repeats for nonnumeric input or a number outside 0 through 7, just like getCommand in Userstate. Returns -1 when getToken returns null (end of input); run then returns.

private int getNumber(String prompt); Same idea as Userstate.getNumber: repeats until the token is an integer. I also require a value greater than zero because it is only used for quantities. Returns -1 on end of input.

private boolean yesOrNo(String prompt); Same as yesOrNo in the sample: true only for an answer starting with Y or y. A null token counts as no.

public void help(); Prints the eight commands and their numbers, like Userstate.help(). Shown when the menu starts and when the user enters 7.

Client operations

Each operation first reads String clientID = context.getClientID(). This replaces LibContext.instance().getUser() in the library sample. Operations finish in this same menu.

public void showClientDetails(); Calls warehouse.getClient(clientID) and prints the Client (ID, name, address, phone, balance). A positive balance is money owed and a negative balance is credit, which is how the backend already stores it.

public void showProducts(); Iterates warehouse.getProducts() and prints product ID, name, and sale price for each product using the getters. I do not print the stock quantity here; the handout asks for price only in the client menu, while the clerk listing also shows quantity. Prints a message for an empty inventory.

public void showTransactions(); Corresponds to getTransactions in Userstate. Calls warehouse.getOrders(clientID) and prints each Order with its OrderItems and total, then the current balance. The library version asks for a date; our backend orders have no date, so I do not prompt for one. Payments only change the balance in the backend, so they appear through the balance line. Prints "No transactions" when the iterator is empty.

public void addToWishlist(); Follows the loop in issueBooks from Userstate: prompt for a product ID, read the quantity with getNumber, call warehouse.addToWishlist(clientID, productID, quantity), report success or failure, then ask "Add more items?" with yesOrNo. An unknown product ID prints an error and nothing changes. Adding a product already in the wishlist replaces its quantity (existing backend rule).

public void displayWishlist(); Calls warehouse.getWishlist(clientID) and prints each WishlistItem (product ID, name, quantity). Prints "Wishlist is empty" when there are no items.

public void placeOrder(); Displays the wishlist, then asks "Place order for all items?". On yes it calls warehouse.processOrder(clientID) and prints the OrderItems, order total, and updated balance. The backend fills what is in stock and moves any shortage to the product waitlist, so "buy everything if available" from the test sequence needs no extra UI logic. A null result (empty wishlist) prints "Order could not be processed". Unlike Project 1, I do not ask for a client ID here.

Navigation methods

public void logout(); Calls context.logout() and nothing else. In the library FSM, Userstate.logout() itself checks getLogin() and chooses changeState(0) or changeState(1). In our design that choice belongs to Context, which uses the stored loginRole and Person 1's matrix: a direct client login goes back to OPENING; a client reached through a clerk (or through manager to clerk) goes back to CLERK. Context also clears the client ID.

public void run(); Sets context = WarehouseContext.instance() and warehouse = context.getWarehouse(), shows help(), and then loops on getCommand(), dispatching with a switch just like process() in Userstate. On LOGOUT it calls logout() and returns to the Context dispatch loop. On -1 it returns without logging out because Context has already stopped. Unlike the sample, run() never calls another state, so switching menus does not build nested run calls.

Command summary

No.	Command	Method	Backend call	Next state
0	Logout	logout()	none	Context decides (OPENING or CLERK)
1	Show client details	showClientDetails()	getClient(id)	CLIENT
2	Show products with price	showProducts()	getProducts()	CLIENT
3	Show transactions	showTransactions()	getOrders(id)	CLIENT
4	Add to wishlist	addToWishlist()	addToWishlist(id, pid, qty)	CLIENT
5	Display wishlist	displayWishlist()	getWishlist(id)	CLIENT
6	Place an order	placeOrder()	processOrder(id)	CLIENT
7	Help	help()	none	CLIENT

Design checks

I will verify these against the handout's test sequence: C1 reached through the clerk adds 5 each of P1, P3, P5 and logout returns to the clerk; the clerk then becomes C2 and C2's wishlist is separate from C1's; C3 logged in directly returns to OPENING on logout; C2's and C3's orders change their balances and the clerk's client list shows the new balances. I also check that a bad product ID or a nonpositive quantity leaves the wishlist unchanged and that an invalid command keeps me in the client menu.

References

LibraryFSM template: samplecode/Userstate.java, LibContext.java, LibState.java (github.com/rsarnath/LibTextFSM). Person 1 design: WarehouseFSM/Person1_Class_Designs.pdf (WarehouseContext, WarehouseState). Backend: abelSam10/430Project1FinalGroup9, WarehouseFinal/Warehouse.java. Requirements: Project 2 Implementation Stage 1 Fall 26 handout.